

Porting Unikraft to Raspberry Pi 5

Semester Project

Lorin Urbantat

August 2026

Supervisor: Clément Thorens

Professor: Prof. Dr. Shweta Shinde

Secure & Trustworthy Systems Group, ETH Zurich

Abstract

Unikraft is a modular unikernel development kit for constructing specialized and lightweight operating-system images. However, Unikraft only offers platform support for KVM and Xen at the moment. To increase its use on accessible, contemporary Arm hardware, we present a new Unikraft platform that boots on the Raspberry Pi 5 and provides the low-level services required to execute Unikraft applications. We handle the AArch64 exception-level transition, discover non-contiguous RAM from the device tree, and replace the bootstrap memory mappings with Unikraft’s managed page tables. We also initialize a PL011 console, integrate the GICv2 interrupt controller and Arm generic timer, and provide a mechanism for repeatedly loading kernels through OpenOCD and GDB without power-cycling the board.

On physical hardware, we verified memory allocation across the Raspberry Pi 5’s complete 4 GiB RAM range. We ran a dynamic paging test that mapped a page at runtime, changed its permissions, translated its virtual address, and unmapped it successfully. In further tests, we completed 1000 consecutive 10 ms scheduler sleeps, performed repeated warm kernel reloads, booted C and C++ Hello World applications, and controlled an LED through GPIO. Our platform establishes a functional basis for running and developing Unikraft applications on the Raspberry Pi 5. We separate platform-specific functionality from the Unikraft core. We currently support only single-core execution and a restricted set of peripherals.

Contents

Abstract	1
1 Introduction	1
1.1 Contributions and scope	1
1.2 Structure of this document	2
2 Background	3
2.1 Unikraft	3
2.2 Related systems	3
2.3 Raspberry Pi 5	4
2.4 Firmware handoff and AArch64 execution state	5
2.5 Runtime hardware description	5
2.6 Development and debug paths	6
3 Design and implementation	7
3.1 Relationship to the Raspberry Pi 3 and common Arm platforms	7
3.2 Boot sequence	8
3.3 MMU setup	8
3.4 Console, interrupts, and time	9
3.5 GPIO design	10
3.6 Cache-safe warm reload	10
3.7 Implementation details	11
4 Evaluation	14
4.1 Hello world	14
4.2 Memory test	15
4.3 Dynamic paging	15
4.4 Interrupt and timer behavior	15
4.5 GPIO output	16
4.6 Warm reload	16
5 Discussion and limitations	17
5.1 Platform scope	17
5.2 Peripheral support	17
5.3 Unsupported execution features	17
5.4 Future work	18
6 Conclusion	19
References	20
A Appendix	22
A.1 Reproduction Guide	22
A.2 Debugging Workflow	24
Use of Artificial Intelligence	24

1 Introduction

Unikernels combine an application with only the operating-system components it requires. By removing services and abstractions that are irrelevant to a particular workload, they can produce compact system images with a small trusted computing base. Unikraft provides these components as a modular development kit, but using the resulting images on a new machine still depends on a platform implementation. The platform must translate the assumptions of the Unikraft core into the boot, memory, interrupt, timer, and console facilities that the target hardware provides.

At present, Unikraft primarily targets virtualized environments such as KVM and Xen. These platforms are suitable for cloud deployment, but they offer a limited path for experimenting with Unikraft on accessible, contemporary Arm hardware. The Raspberry Pi 5 is a useful target for this purpose: it is widely available, uses a modern 64-bit Arm processor, and exposes the low-level mechanisms required by a small operating system. At the same time, it is not a trivial target. Its BCM2712 system-on-chip, firmware boot protocol, physical memory layout, interrupt controller, and debug behavior impose constraints that are absent from a virtual machine.

In this work, we develop a Unikraft platform for the Raspberry Pi 5. Execution enters at EL2 and the platform then drops down to EL1 as required by Unikraft. It creates an initial identity-mapped address space, discovers potentially non-contiguous RAM that the firmware advertises through the device tree, and passes valid memory descriptions to Unikraft’s common paging and allocation code. Unikraft’s common paging subsystem replaces the bootstrap tables and provides runtime page mapping, unmapping, permission changes, and address translation. The platform also initializes a PL011 UART for early diagnostics and standard console output, connects the GICv2-compatible interrupt controller, and uses the Arm non-secure physical generic timer for scheduler wake-ups. It provides basic access to the GPIO pins through the RP1 I/O controller. We keep platform-specific code separate from the Unikraft core and integrate it through the existing build and configuration mechanisms.

Low-level development also requires a short and repeatable test cycle. We therefore provide a warm-reload mechanism that replaces the kernel in RAM without power-cycling the board. Unlike a cold boot, a warm reload begins after the previous kernel has enabled the MMU and caches. Dirty data-cache lines may not yet have reached RAM, while the instruction cache and address-translation state may still refer to the old kernel image. A resident reload trampoline prepares the processor before GDB loads the replacement kernel and device tree. It cleans and invalidates the data caches, invalidates the instruction cache and translation lookaside buffer, and then disables the caches and MMU. This restores the processor state expected by the new kernel and enables repeated deployment through OpenOCD and GDB without a power cycle.

1.1 Contributions and scope

Our principal contribution is a functional single-core Raspberry Pi 5 platform that boots Unikraft applications on physical hardware. We tested memory allocation, memory integrity, runtime page mapping, and page protection on the board. We also completed 1000 consecutive scheduler sleeps of 10 ms and booted hello-world applications written in C and C++. The platform provides basic GPIO direction, level, and pull control. We leave multicore operation and broader peripheral support outside the present scope.

1.2 Structure of this document

Chapter 2 introduces the relevant Unikraft interfaces and Raspberry Pi 5 hardware requirements. Chapter 3 describes the platform design and implementation. Chapter 4 evaluates boot, memory, timing, and reload behavior on the board. Chapter 5 discusses limitations and future work, Chapter 6 concludes the report, and the appendices provide reproduction and configuration details.

2 Background

When porting Unikraft to a physical machine, we must configure the hardware-specific features according to Unikraft’s expectations. The platform must receive control from the machine’s firmware, establish the execution environment expected by the common Unikraft code, and connect architectural services to the concrete devices described by the hardware. This chapter first defines what Unikraft is, introduces the unikernel landscape, and then introduces the Raspberry Pi 5 boot and peripheral environment.

2.1 Unikraft

Unikraft decomposes operating-system functionality into independently configurable micro-libraries with explicit interfaces. Its build system combines an application, selected libraries, an architecture, and exactly one target platform into a single image [10]. The Raspberry Pi 5 is built around Broadcom’s BCM2712 system-on-chip, which contains four 64-bit Arm Cortex-A76 cores [5]. Developers can maintain a platform separately from the parent source tree and supply it to the build as an external platform. This separation keeps Raspberry Pi 5-specific requirements, including its firmware handoff, memory layout, interrupt controller, timer, and peripheral mappings, out of common Unikraft components.

The first boundary is the boot handoff. A platform constructs a persistent `ukplat_bootinfo` description and calls `uk_boot_early_init()` before transferring control to the non-returning `uk_boot_entry()`. A device tree describes the machine’s processors, memory, and peripherals as a hierarchy of nodes and properties. Its binary representation is called a device-tree blob (DTB), whose address the firmware passes to the kernel. Unikraft’s boot information needs to include a pointer to this DTB, the boot arguments from `/chosen/bootargs`, and typed memory-region descriptors. Before this transfer, our Arm platform must also install the exception-vector table, initialize Unikraft’s logical-CPU representation of the boot processor, make usable memory available to the common allocator, and initialize the interrupt controller required by the timer. A registered console driver provides console output, while the platform time interface supplies timer initialization, an interrupt identifier, counter ticks, and monotonic time. The Raspberry Pi 5 port must expose the firmware-provided boot information and board-specific hardware through Unikraft’s common interfaces, while leaving higher-level boot, memory allocation, scheduling, and console logic to Unikraft.

2.2 Related systems

Unikernels combine an application with a specialized operating-system image, commonly linking the application with the system facilities needed for its deployment. Projects applying this idea differ in their programming models, compatibility goals, and execution environments. MirageOS constructs standalone systems primarily from OCaml libraries and uses OCaml’s static type system and typed module interfaces to enforce invariants between components [20]. IncludeOS is a library operating system implemented in modern C++ that builds single-application images for C and C++ applications [21]. OSv instead implements a substantial, though incomplete, Linux-compatible ABI to run existing applications and language runtimes inside virtual machines [22]. Solo5 places a small guest-facing interface between a unikernel and its host or hypervisor; its tenders and bindings mediate access to the underlying execution environment [8].

Unikraft combines application compatibility with fine-grained specialization through a micro-library architecture. Unikraft divides operating-system facilities into selectable components, while the system exposes both compatibility interfaces and composable internal APIs intended for performance-sensitive code [7]. Developers can compile and link applications with Unikraft using their existing build systems, and supported x86-64/KVM configurations can also execute suitable Linux binaries through a binary-compatibility layer. Build-time configuration selects facilities such as the scheduler, allocator, filesystem, and network stack. In contrast to MirageOS’s OCaml-centred native programming model, Unikraft accommodates applications and runtimes written in several languages through conventional C and POSIX interfaces. Compared with OSv’s broad Linux-compatible ABI, its micro-library organization provides a more explicit mechanism for omitting or replacing OS components. Unikraft also separates application-facing libraries from platform-specific implementations. This boundary is central to the present work: the Raspberry Pi 5 platform supplies boot, memory, interrupt, timer, and console functionality behind Unikraft interfaces without requiring changes to application-facing libraries.

Within Raspberry Pi development, the closest Unikraft predecessor is the external Raspberry Pi 3 platform [6]. It demonstrates firmware-loaded execution and implements board-specific console, interrupt, timer, mailbox, and display support. Its code is not directly portable to the Raspberry Pi 5: the BCM2712 changes the boot environment, memory topology, interrupt integration, and peripheral layout. This project therefore follows the predecessor’s external-platform boundary while replacing the board-specific mechanisms.

Other bare-metal Raspberry Pi projects provide useful implementation references but solve a different problem. Circle is a mature C++ environment with Raspberry Pi 5 support and a much wider driver set, including multicore execution, USB, storage, networking, and wireless facilities [2]. HobOS is a smaller hobby operating system for the Raspberry Pi 5 whose focused boot and UART code is valuable during early bring-up [3]. Minimal UART examples serve a similar diagnostic role [4]. These works show how to operate the board directly, whereas a Unikraft platform must additionally translate each mechanism into Unikraft’s common boot, allocator, interrupt, scheduler, time, and device interfaces. These serve as valuable points of reference for implementing the Raspberry Pi 5 platform for Unikraft.

2.3 Raspberry Pi 5

The Raspberry Pi 5 combines Broadcom’s BCM2712 application processor with Raspberry Pi’s RP1 I/O controller. The BCM2712 contains four 64-bit Arm Cortex-A76 cores and connects to LPDDR4X memory. It also provides the architectural components used by this platform, including the Arm generic timer and the GIC-400-compatible interrupt controller [5].

RP1 provides most of the board’s external I/O, including GPIO, UART, SPI, I2C, PWM, USB, and Ethernet. It is a PCIe endpoint connected to the BCM2712 through a four-lane PCIe 2.0 link [9] [24]. The firmware configures this link and assigns RP1 a base address register (BAR). CPU accesses to addresses within that BAR become PCIe transactions to RP1, even though software uses ordinary memory-mapped I/O instructions.

The board has two PCIe connections that are relevant to platform software. The internal four-lane link connects the BCM2712 to RP1, while a separate single-lane PCIe 2.0 link reaches the external FFC connector [5]. This project relies on the firmware-configured RP1 link for UART and GPIO access. It does not initialize either PCIe controller or support devices attached to the external connector.

The Raspberry Pi 5 also has a three-pin debug connector that can carry either the dedicated debug UART or Serial Wire Debug (SWD). This project configures the connector for SWD and

uses an RP1 UART on GPIO 14 and GPIO 15 for console output. This arrangement keeps the serial console available while a debugger controls the processor.

We connect both interfaces through a Raspberry Pi Debug Probe. The probe presents the UART as a USB serial port and presents SWD through the CMSIS-DAP protocol [27]. SWD lets OpenOCD and GDB stop and resume the boot core, inspect registers and memory, set breakpoints, and load a kernel into RAM.

2.4 Firmware handoff and AArch64 execution state

In this project, we use the 4 GiB variant of the Raspberry Pi 5. Arm defines four exception levels: applications normally use EL0, operating-system kernels use EL1, hypervisors use EL2, and secure monitors or firmware use EL3 [11]. Unikraft’s native Arm path executes as a kernel at EL1. The Raspberry Pi firmware transfers control to the kernel in EL2, so the platform must configure the EL2 controls needed by EL1, arrange an exception return to EL1, and install an EL1 vector table before the kernel can handle exceptions safely. Other exception levels such as EL3 are not involved in the kernels’s boot process. The current scope is single-core; non-boot cores must not proceed through the same initialization concurrently.

The boot image is a raw, fixed-address AArch64 image named `kernel_2712.img`, the Pi 5 firmware’s model-specific default kernel name [12]. The adopted AArch64 handoff follows the conventional register contract in which `x0` contains the physical address of the DTB, with the MMU disabled at entry [13]. The entry code must preserve the DTB pointer while it establishes a stack, exception state, and the address space required to execute C code. A debugger-loaded image must recreate the same contract.

2.5 Runtime hardware description

On Raspberry Pi systems, the firmware selects a base DTB for the detected board, applies `config.txt` parameters and overlays, and passes the resulting hardware description to the kernel [14]. The DTB available at run time may therefore differ from one stored with the source code. This project observed the practical effect during bring-up: an early captured description exposed only 640 MiB, whereas the live 4 GiB firmware description contained two enabled RAM ranges separated by a 4 MiB hole. Compiling a single contiguous RAM size into the platform would either discard usable memory or allow allocation from an absent range.

We therefore parse the runtime hardware description twice. Early boot must discover enough enabled RAM to construct safe initial mappings without relying on facilities that themselves require mapped memory. The present scope also assumes the board’s known early UART location without parsing the device tree, while the platform discovers memory and interrupt topology at run time.

2.5.1 Memory-mapped devices

Peripherals use memory-mapped I/O (MMIO), whose ranges the platform must map as device memory to preserve register semantics. The bootstrap address space must therefore cover its code, data structures, DTB, and peripheral registers while distinguishing RAM from the device aperture.

The console is a PL011-compatible UART [15]. A polling path provides early diagnostics before interrupts are available and later supplies Unikraft’s standard and emergency output. The firmware DTB also identifies a GIC-400 interrupt controller, whose GICv2 interface translates

DT interrupt specifiers and dispatches interrupts [16]. Finally, the Arm generic timer supplies monotonic time and scheduler deadlines through its system counter, comparator, and DT-described interrupt [17].

2.6 Development and debug paths

UART provides execution traces, while SWD enables kernel reloads, core control, as well as memory and register inspection. OpenOCD exposes the BCM2712 debug target to GDB, which uses the unstripped ELF for source-level control and image loading [18]. Loading the DTB separately and restoring its address in `x0` preserves the firmware boot ABI.

Reliable warm reloads additionally require the running kernel to enter a known identity-mapped state with the MMU and caches disabled before GDB replaces the image and DTB. Otherwise, dirty data, stale instructions or translations, and differing address spaces can corrupt the new execution. Chapter 3 describes the resulting reload mechanism.

3 Design and implementation

This chapter presents the design and implementation of the native AArch64 platform that enables Unikraft to run on the Raspberry Pi 5. It focuses on the mechanisms that transform the limited firmware handoff into the execution environment expected by Unikraft: a valid address space, an accurate description of usable memory, console output, interrupt delivery, timer-driven scheduling, GPIO access, and a repeatable development workflow.

The platform reuses Unikraft’s common boot, allocation, device-tree, interrupt-controller, and console interfaces, while board-specific code provides entry, address-space, UART, timer, and setup functionality. A central design choice is to use the fixed physical load address 0x80000 provided by the Raspberry Pi firmware. Avoiding runtime relocation simplifies the early boot path. The entry code verifies this assumption by comparing its runtime and link-time addresses and parks the processor if they do not match. The developer can then inspect both addresses with GDB.

The chapter first compares this structure with the existing Raspberry Pi 3 platform and Unikraft’s common Arm implementation. It then follows the platform from firmware entry through bootstrap memory discovery and normal Unikraft initialization, before describing console, interrupt, timer, GPIO, and warm-reload support. The final part relates these design decisions to the source layout, build integration, and validation applications.

3.1 Relationship to the Raspberry Pi 3 and common Arm platforms

We used the existing external Raspberry Pi 3 platform as a reference for firmware-loaded execution, but we did not adopt its structure directly [6]. It encodes much of the board description in hard-coded platform code, does not preserve the firmware DTB through early boot, and implements several facilities locally that common Unikraft interfaces now provide. We retain its single-core policy: as on the Raspberry Pi 3 platform, we park the secondary processors while one boot processor performs initialization.

The Raspberry Pi 5 platform is therefore structurally closer to Unikraft’s common Arm and KVM/Arm paths. It preserves the DTB pointer that the firmware supplies in `x0`, constructs the normal boot-information representation, and reuses common boot, allocation, device-tree, interrupt-controller, and console abstractions. This reduces duplicated platform code and keeps Raspberry Pi-specific behavior at the hardware boundary. The common KVM/Arm path is nevertheless more general because it supports position-independent executables and runtime relocation. Such an executable can run at an address other than the one assumed when it was linked. Raspberry Pi firmware always loads this platform’s image at the configured 0x80000 address, and the entry code depends on that fixed address. The build therefore rejects Unikraft’s static position-independent executable (PIE) configuration, `CONFIG_OPTIMIZE_PIE=y`.

Peripheral differences also prevent direct reuse of the Raspberry Pi 3 implementation. The Raspberry Pi 3 platform implements the VideoCore mailbox protocol to exchange property messages with the firmware. It also configures its SoC GPIO routing and UART parameters itself. On the tested Raspberry Pi 5 setup, the PL011-compatible UART controller resides in RP1. The CPU accesses this controllers memory-mapped registers through the internal PCIe link and the RP1 BAR configured by the firmware. RP1 therefore implements the UART peripheral.

The firmware also configures the UART's pin routing, clock, and baud rate. The Raspberry Pi 5 boot, console, interrupt, and timer paths obtain the remaining required information from the DTB. They do not use the Raspberry Pi 3 mailbox implementation.

3.2 Boot sequence

The kernel entry happens in `entry64.S`. Here the firmware passes the DTB address in `x0`. The entry code preserves this value before changing processor state. If execution begins at EL2, it configures AArch64 execution at the lower exception level, permits EL1 access to the physical counter and timer, clears the virtual counter offset, and returns to EL1. Entry at EL1 follows the same remaining path. The entry code parks all secondary cores according to their affinity identifier, so later initialization is deliberately single-core.

After checking the static load address against the link-time address, the boot CPU installs the EL1 vector table and may wait in an early debugger spin loop. We place this loop before the rest of the setup so that we can step through and debug the kernel from its first operations. After the GDB spin loop, the boot CPU selects a statically reserved 4 KiB boot stack and invokes the MMU bootstrap while the DTB pointer is still live. Only after the MMU bootstrap enables translation and caches does the boot CPU clear BSS, initialize the early UART, install the per-CPU state base address in `TPIDR_EL1`, and construct the normal Unikraft boot-information structure from the DTB. The entry code then transfers control to C in `setup.c`. The C platform entry sets up the UART as Unikraft's standart console, reserves the null page, and calls `uk_boot_early_init()` coalesce the memory-region list. It then calls `ukplat_mem_init()`, which replaces the bootstrap tables with a managed page table. After applying device-memory attributes, the platform initializes GPIO, the interrupt controller, and the boot CPU before entering the application through `uk_boot_entry()`.

This order supplies a stack and address space before clearing BSS, enables C, and initializes the controller and boot CPU before interrupt-driven time. UART starts early, keeping later failures observable.

3.3 MMU setup

The DTB describes the available RAM regions. On our Pi 5, the relevant entry has the following form:

```
memory@0 {
    device_type = "memory";
    reg = <0x00 0x00 0x00 0x3fc00000 0x00 0x40000000 0x00 0xc0000000>;
};
```

The firmware encodes each memory region as `<address-high address-low size-high size-low>`. Each 32-bit hexadecimal value represents one half of a 64-bit value. The entry above describes two memory regions: one starts at `0x0000000000000000` with size `0x000000003fc00000`, and the other starts at `0x0000000040000000` with size `0x00000000c0000000`. We must parse the device tree to obtain this information before we set up the MMU. However, Unikraft's complete FDT library requires an active MMU. We therefore implement a small, bounded parser in assembly that reads only the data required for MMU setup. We parse the FDT a second time after bootstrapping the MMU, but this approach lets the same kernel support every Pi 5 memory configuration.

Our parser interprets each non-zero memory tuple as the address range `[start, start + size)`: the range includes the byte at `start` but excludes the byte at `start + size`. It rounds valid

ranges outwards to 1 GiB boundaries and identity-maps them as Normal write-back, write-allocate, inner-shareable blocks. Mapping memory in 1 GiB blocks lets the bootstrap use a fixed page-table structure that does not depend on the Raspberry Pi 5's installed memory size or layout. The MMU uses a 48-bit virtual and physical address geometry with 4 KiB pages, so translation starts at level zero. The bootstrap fills only two entries of the level-zero root table. Root entry zero points to one level-one table whose 512 block descriptors cover the entire lower address space: DT-reported RAM becomes identity-mapped Normal write-back, write-allocate blocks ($PA == VA$), and everything else stays invalid at this level. Root entry 511 points to a second level-one table that identity-maps physical addresses 0 through 512 GiB as Normal WBWA again, but places them at the top of the accessible space, beginning at `0x0000FF80_00000000`. This window coincides with Unikraft's Arm64 direct-map area. Unikraft's native paging code reaches any physical frame, including the managed page tables it builds, by computing $VA = PA + 0xFF80_00000000$, so this table must already be present when paging construction starts. This coarse bootstrap map is safe because Unikraft scans the FDT again and allocates memory only from regions that exist.

The bootstrap independently installs the BCM2712 CPU peripheral aperture from 64 GiB through 128 GiB as 1 GiB Device-nGnRnE blocks with privileged and unprivileged execute-never attributes. Everything neither reported as RAM nor included in this fixed aperture remains invalid. Separating the memory types avoids mapping the same physical range simultaneously as cacheable RAM and device memory.

After constructing the page tables, the platform installs the new memory configuration and clears stale address translations and cached instructions before enabling the MMU and caches. We clear them because we observed stale instructions in the cache during development. Since the bootstrap creates an identity map, the kernel continues to execute at the same addresses after translation is enabled.

Once the bootstrap can execute normal C code and use Unikraft's `libfdt`, a second pass constructs the authoritative Unikraft memory-region descriptors. The pass requires at least one RAM range and checks that one range contains the entire loaded Unikraft kernel. The platform reserves one additional page at physical address zero to avoid confusion around NULL pointers. It also describes the BCM2712 peripheral aperture as device memory and coalesces the descriptors before common paging marks free regions as unmapped.

With `LIBUKPAGING` enabled, `ukplat_mem_init()` creates and activates Unikraft's managed page table from these descriptors. RAM uses Normal WBWA attributes. The platform then unmaps and remaps device descriptors as Device-nGnRnE because the generic Arm64 attribute replacement retains the memory-type bits from the initial Normal mapping. GPIO initialization runs only after this replacement, so its MMIO accesses use the final device mapping. For builds that do not use `LIBUKPAGING`, the initial identity mapped page tables are preserved.

3.4 Console, interrupts, and time

The console path uses a minimal polling PL011 driver at a fixed CPU-visible address. The driver accesses the RP1 UART through a firmware-configured PCIe mapping. The firmware also configures the UART clock, baud-rate divisors, and pin routing before it transfers control to Unikraft. The driver preserves these settings and only enables the UART for polling-based I/O. We use this console for boot diagnostics and later register it with `libukconsole` for standard input, standard output, and emergency output. We chose polling because it requires less setup than an interrupt-driven design. Arm specifies the PL011 register interface 15.

Interrupt-controller selection is less board-specific. The firmware DTB contains an `arm,gic-400` controller the necessary information to address it. The platform uses Unikraft's GICv2 driver

which locates the compatible node in the DTB and obtains its register addresses.

We use the Arm generic timer for timekeeping and scheduling. It reads the architectural counter, programs timer deadlines, and controls timer interrupts through standard Arm registers. The platform obtains the timer frequency and interrupt information from the DTB where available. Because the timer is generic, the Arm architecture defines its interface and Unikraft’s existing timer mechanisms work well with it. Thus the platform requires little Raspberry Pi-specific timer code.

3.5 GPIO design

The GPIO controller for the Raspberry Pi 5’s 40-pin header resides in RP1 rather than in the BCM2712. RP1 is a PCIe endpoint connected to one of the BCM2712’s PCIe root complexes. During boot, the firmware brings up this internal PCIe link, assigns RP1’s BAR, and configures an outbound address window in the root complex. The `pciex4_reset=0` setting tells the firmware to preserve this configuration when it transfers control to Unikraft.

RP1’s registers therefore appear at fixed physical addresses in the CPU’s address space. A CPU load or store to one of these addresses reaches the PCIe root complex, which converts the access into a PCIe memory transaction to RP1. The firmware’s PCIe configuration makes the RP1 registers accessible on the CPU through ordinary MMIO instructions.

The platform relies on this firmware handoff. It does not initialize the PCIe root complex, enumerate RP1, assign its BAR, or configure the address window. Implementing these operations would remove the dependency on `pciex4_reset=0` and allow the platform to change the RP1 configuration. The current GPIO implementation does not need to perform them because the firmware has already completed them.

The GPIO interface supports pins 0 through 27. It separates direction from output level, so an application first selects input or output and then reads or writes the pin. Pull-up, pull-down, and no-pull settings are also available. Raspberry Pi’s `pinctrl` utility implementation, Linux RP1 pin-control driver, and The RP1 peripherals datasheet, guided our implementation for function selection, pad configuration, and GPIO access sequencing [25][26][24].

3.6 Cache-safe warm reload

A warm reload begins after the previous kernel has Enabled MMU and caches. Dirty data-cache lines may not yet have reached RAM, and the instruction cache and TLB may still refer to the old kernel. If GDB overwrites the image in this state, the processor may execute stale instructions or write old data over the replacement image.

The platform therefore keeps a reload trampoline at the fixed address `0x200000` (`reload64.S`). Before GDB overwrites the old image, it executes this trampoline. The trampoline cleans and invalidates the data caches, invalidates the instruction cache and EL1 TLB, disables EL1 address translation and caches, and stops at the fixed park address `0x200400`.

GDB checks for the trampoline before each load (`ConnectJTAG.gdb`). If its signature is present, GDB runs the trampoline and verifies that execution reached the park address with `SCTLR_EL1.M`, `SCTLR_EL1.C`, and `SCTLR_EL1.I` cleared. If the signature is absent, the script accepts the target only when it is running at EL2. This exception permits the initial load because the Raspberry Pi firmware enters the kernel at EL2 and no previous kernel state must be removed. A missing signature at EL1 indicates an unsupported warm state, so GDB rejects the load. After a successful load, GDB places the DTB at `0x10000000`, passes its address in `x0`, and starts the kernel at its fixed entry address.

3.7 Implementation details

The platform implementation resides in `plat/native/rpi5`. `Config.uk` selects the required modular Unikraft components and provides an option for the early GDB spin loop. `Makefile.uk` builds the Raspberry Pi-specific sources together with reused code from Unikraft's `common` and `kvm/arm` code. Because the linker places the image at a fixed address, the build rejects position-independent configurations and `Linker.uk` selects `_librpi5plat_entry` as the ELF entry point. The top-level `Makefile.rpi5` provides a reproducible build using `configs/rpi5_defconfig` and, by default, the catalog `native/helloworld-c` application.

3.7.1 Entry path and linker layout

`link64.lds.S` defines the fixed image and the regions that the linker reserves for warm reload, with link-time assertions preventing overlap and excessive image growth. `entry64.S` implements the corresponding entry path; it prepares EL1 execution and the bootstrap environment before calling the C platform entry in `setup.c`. The preceding boot-sequence section describes the ordering and responsibilities of this path.

3.7.2 Bootstrap MMU

`rpi5_start_mmu` constructs a sparse identity map using 4 KiB translation tables and 1 GiB block descriptors. The 48-bit geometry starts translation at level zero. Root entry zero covers the lower 512 GiB, and root entry 511 connects the upper-half direct map required during Unikraft's managed page-table construction. The bootstrap maps the fixed BCM2712 device aperture with Device memory attributes. The entry path requires disabled stage-1 translation and data cache; a violation reports error 11 through `x0` and parks the processor. The bootstrap stores other parser and mapping failures in `rpi5_mmu_error` before the processor parks. Because the lower mapping is identity-based, enabling the MMU does not require runtime relocation of the kernel.

3.7.3 Boot information

Before boot, the build records which addresses contain the Unikraft binary. The linker symbols `__BASE_ADDR` and `__END` mark the first address and the end of this binary. At boot, the DTB supplies a list of the Raspberry Pi's RAM ranges. `rpi5_memory_mrds` reads every range and marks the memory that the binary does not occupy as available for allocation. For the range that contains the binary, this means marking the memory before `__BASE_ADDR` and after `__END` as available. The existing record for the binary keeps the memory between these addresses reserved.

The function stops the boot if a DTB range overlaps only part of the binary. Such a range would describe the binary as partly outside RAM. A valid DTB lists non-overlapping RAM ranges, so only one range can contain the binary. Unikraft's shared FDT code adds separate records for the DTB and the initial RAM disk, and it reads the kernel command line.

The platform builds exact memory-region descriptors from the DTB, reserves page zero to avoid null-pointer confusion, and adds the 64–128 GiB BCM2712 peripheral aperture as device memory. It calls `uk_boot_early_init()` before common memory initialization, so descriptor coalescing occurs before paging marks free regions as unmapped. `ukplat_mem_init()` then creates the managed table when `LIBUKPAGING` is enabled. `rpi5_paging_remap_device_memory()` reapplies Device-nGnRnE attributes after the table switch.

3.7.4 Console, interrupts, and time

`pl011.c` supplies polling UART for early diagnostics and later registers the same device with `libukconsole` for standard and emergency streams. It configures only the UART mode and transmit/receive state, while the firmware supplies clock, baud rate, and pin routing.

The common GICv2 driver, selected through `LIBUKINTCTLR_GICV2`, provides interrupt-controller support. The board-specific timer glue in `time.c` locates the architectural timer node, selects the non-secure physical-timer PPI, and registers the handler through the common interrupt-controller interface. We derive most of `generic_timer.c` from `common/arm/generic_timer.c`. It uses the physical counter and timer registers consistently and derives its conversion factor from the DTB.

3.7.5 RP1 GPIO

`rpi5_gpio.c` implements the platform-local GPIO API declared in `rpi5_gpio.h`. Platform setup calls `rpi5_gpio_init()` after common memory initialization and the device-memory remap. Initialization marks the fixed firmware BAR as available and initializes the GPIO spin-lock, which ensures no concurrent accesses to GPIO registers.

`rpi5_gpio_direction()` validates the pin number and direction, updates the RIO output-enable register, and configures the corresponding pad. Input mode enables the input path and disables the output path. Output mode enables the input path and output driver. The function then selects the GPIO function and clears output and output-enable overrides. `rpi5_gpio_get()` reads the RIO input register, while `rpi5_gpio_set()` writes the output set or clear alias. `rpi5_gpio_set_pull()` changes only the pad pull field. Invalid pins, directions, pull modes, and null read pointers return errors.

3.7.6 Reloading a running target

`reload64.S` implements the resident stackless trampoline. It masks exceptions, cleans and invalidates every data or unified cache by set and way, invalidates the instruction cache and EL1 TLB, disables translation and caches, and performs the additional physical-range invalidation required by the observed BCM2712 debug-access-port behavior. It then branches to the reserved park address where GDB has a breakpoint.

`debug_cfg/ConnectJTAG.gdb` exposes this sequence as `rpi5_reload`. The command first validates the DTB size and reads the resident trampoline signature. This signature is an instruction-byte marker rather than a cryptographic signature. If the signature matches, the command runs the old image's trampoline and verifies both the park address and the cleared `SCTLR_EL1` state before loading the replacement ELF and DTB.

If the signature is absent, the command permits the load only at EL2. This test distinguishes the expected firmware entry from the normal warm Unikraft state because the platform drops from EL2 to EL1 at the start of kernel execution. However, EL2 alone does not prove that the processor is at a cold firmware halt. An early kernel halt or another EL2 program could satisfy the same test. A stronger check would also verify that EL2 translation and caches are disabled and, if available, that the program counter identifies the expected firmware halt. In practice however this was unnecessary.

The trampoline entry and park use fixed numerical addresses because GDB resolves symbols against the incoming image. Their addresses must remain identical between the running and replacement images.

3.7.7 Validation

The repository includes a `testapps` directory with six small Unikraft applications. Keeping these applications beside the platform makes each evaluation build reproducible without depending on an external catalog of applications. `hello-world` and `hello-world-cpp` exercise boot and console output with C and C++, respectively. `eval-mem` allocates almost all reported memory in chunks, writes and verifies a deterministic word pattern, and fails if capacity falls below the threshold needed to include the second RAM bank. `dynamic-paging` maps an allocated page, checks read-only and read/write permissions, verifies virtual-to-physical translation, unmaps the page, and confirms through `libuknefault` that the address is inaccessible. `eval-timer` checks monotonic-clock progression and performs 1000 scheduler sleeps with a minimum-duration assertion. `rpi5-gpio` configures GPIO23 as an output and blinks a connected LED ten times. We present their measured results and build commands in the evaluation chapter and appendix.

4 Evaluation

The evaluation tests six platform properties on a 4 GiB Raspberry Pi 5: complete boot sequence, memory allocation, dynamic paging, scheduler timing, GPIO output, and warm kernel reload. The tests establish functional correctness for the exercised single-core configuration. They do not measure performance or long-term reliability.

Property	Success criterion
Boot	A clean build reaches the application and the error-zero halt path.
Memory	1020 chunks of 4 MiB of Memory are addressable and allocatable by an application.
Dynamic paging	A page can be mapped, protected as read-only and read/write, translated to its physical address, and made inaccessible by unmapping it.
Timer	1000 sequential 10 ms sleeps complete without an early wake-up or counter regression.
GPIO	An LED connected to GPIO23 blinks ten times in sequence.
Warm reload	GDB replaces the running kernel and DTB in RAM twice without rebooting or power-cycling the board. The reload command rejects invalid processor states and a DTB or kernel image that exceeds its reserved memory region.

4.1 Hello world

We first booted simple hello-world applications written in C and C++. The C application is:

```
#include <stdio.h>

int main(void)
{
    puts("Hello, World!");

    return 0;
}
```

The C++ application uses `extern "C" int puts(const char *message);` to declare the C linkage of `puts`, then produces the same output.

We packaged both applications with Unikraft's build tools and produced kernel images for the Raspberry Pi 5. The board booted both images. Each application printed `Hello, World!` to

the console and returned zero. These results verify that the kernel loads correctly, sets up the system, and transfers control to either a C or C++ application.

4.2 Memory test

We next tested memory allocation and the MMU. The test allocates 1020 chunks of 4 MiB, writes a signature to each chunk, and reads the signatures back for verification.

The test allocated and verified memory across the complete 4 GiB RAM range. This result shows that our memory initialization and MMU setup cover both firmware-described RAM banks.

4.3 Dynamic paging

We tested the managed page table with the `dynamic-paging` application. The test first uses `uk_nofault_probe_r()` to confirm that virtual address `0x200000000000`, or 2 TiB, is unmapped. It then asks `uk_paging_page_map()` to allocate a physical frame and map one read-only page at that address. A no-fault read succeeds, while a no-fault write fails.

The test changes the mapping to read/write with `uk_paging_page_set_attr()`, writes a 64-bit value, and reads it back. It also checks that `uk_paging_virt_to_phys()` returns a physical address. Finally, it calls `uk_paging_page_unmap()` and confirms that a no-fault read fails again. All checks passed on the Raspberry Pi 5. This result verifies page-table allocation, runtime mapping, permission changes, address translation, TLB-visible unmapping, and fault recovery for the tested page.

4.4 Interrupt and timer behavior

The timer test uses the GIC-400 controller selected from the DTB and the non-secure physical-timer PPI. It calls `uk_sched_thread_sleep()` 1000 times with a requested interval of 10 ms. The test fails an iteration if the clock regresses or the elapsed time falls below 10 ms.

```
int main(void)
{
    unsigned int i;

    for (i = 0; i < ITERATIONS; i++) {
        const __nsec before = ukplat_monotonic_clock();
        __nsec after;

        uk_sched_thread_sleep(INTERVAL_NS);
        after = ukplat_monotonic_clock();
        if (after < before || after - before < INTERVAL_NS) {
            printf("rpi5-timer-test: FAIL at %u\n", i);
            return 1;
        }
    }
    puts("rpi5-timer-test: PASS (1000 x 10 ms)");
    return 0;
}
```

All 1000 sleeps completed in about 10.1 seconds. We observed no early wake-up, counter regression, or interrupt storm. Because we set no upper bound, the test does not measure lateness, jitter, interrupt latency, or clock accuracy.

4.5 GPIO output

We configured GPIO23 as an output with the `testapps/rpi5-gpio` application and blinked a connected LED ten times. During development, we connected an ESP32 to the same pin and monitored its level through a serial connection on the development computer.

4.6 Warm reload

Finally, we tested warm kernel reloads. A warm reload replaces the kernel and DTB in RAM and starts the replacement kernel without rebooting or power-cycling the Raspberry Pi. GDB performed this sequence twice in succession. Each reload reached the cache-off park at `0x200400`, loaded the current image and DTB, and resumed execution. Both replacement kernels repeated the timer test, returned zero, and reached normal shutdown.

Overall, we demonstrate single-core boot of C and C++ applications, use of the complete firmware-described RAM region, runtime page mapping and protection, scheduler-timer operation, RP1 GPIO output, and two warm reloads on the tested board.

5 Discussion and limitations

The platform provides a minimal path from Raspberry Pi firmware to a single-core Unikraft application. Its main limitations are narrow peripheral support, no support for PIE, and no support for multicore operation.

5.1 Platform scope

Our implementation assumes the tested Raspberry Pi 5 firmware configuration. We compile in the UART address, select GICv2 and one timer tuple at build time, and use `config.txt` to fix the boot mode, filenames, debug interface, and UART selection. We have not tested compatibility with other firmware releases, memory capacities, or board revisions.

5.2 Peripheral support

We support only the PL011 console, the GICv2 interrupt controller, the ARM architectural timer, and basic RP1 GPIO. The UART driver accesses a fixed MMIO address and uses polling for transmission and reception. The GPIO driver provides direction, read, write, and pull control for GPIO 0 through 27 using a platform-local API.

The GPIO driver depends on the firmware preserving the internal RP1 PCIe link and its fixed BAR. It does not discover or repair this configuration. Ethernet remains unsupported. Adding it would require the RP1 Ethernet MAC and PHY drivers, DMA and interrupt plumbing, as well as a `uknetdev` adapter. The presence of Unikraft’s generic PCI bus code does not make the remaining RP1 devices or the external PCIe connector usable on this platform. The device-memory mapping only makes MMIO addresses addressable; it does not enumerate PCIe devices, assign resources, or configure interrupts.

The on-board Wi-Fi device uses SDIO rather than PCIe. Supporting it would require an SDIO host-controller driver, device-specific initialization and firmware handling, interrupt support, and integration with Unikraft’s networking interfaces. We do not implement this functionality. Applications can use the serial console, timer, and basic GPIO, but network and general PCIe devices require substantial additional platform and driver work.

Compared with the systems discussed in Chapter 2, our platform supports far fewer devices than Circle [2], but it has a similar scope to the Raspberry Pi 3 platform for Unikraft [6]. We contribute a minimal Unikraft platform that boots and executes basic C code, along with practical findings from developing it.

5.3 Unsupported execution features

We link the image for a fixed firmware load address, and the build rejects static PIE. Consequently, a loader cannot place the same binary at an arbitrary address, and our platform does not provide the runtime relocation that the common KVM/Arm path supports.

The bootstrap and managed address spaces retain identity mappings for the kernel and RAM. The paging subsystem can create additional runtime mappings and change their protection, but it does not provide address-space layout randomization or relocate the loaded kernel.

The platform runs only one boot core. The entry code parks the other cores, and we do not implement a secondary-entry protocol, per-CPU setup, inter-processor interrupts, or per-core GIC and timer initialization.

Multicore support remains outside the scope of this project.

5.4 Future work

We identify the following priorities for future work:

1. Add networking through RP1 using the `uknetdev` driver
2. Add proper PCIe support without relying on the firmware configuration. This would also enable using the second external PCIe link that the Raspberry Pi 5 has
3. Add multicore entry, per-CPU state, GIC and timer initialization, inter-processor interrupts, and reload handling for all cores.
4. Add PIE and runtime relocation support.

6 Conclusion

In this work, we implement a Unikraft platform for the Raspberry Pi 5. Our platform boots a Unikraft application on physical hardware, discovers the firmware-described memory, creates bootstrap memory mappings and later replaces them with Unikraft’s managed page tables. It provides serial output and supports interrupts and scheduler timing through the GICv2 controller and Arm generic timer. We also provide direction, read, write, and pull control for GPIO 0–27 through RP1. Our cache-safe reload mechanism enables repeated kernel deployment through OpenOCD and GDB without power-cycling the board.

In our evaluation, we verify complete hello-world boots with C and C++ applications, memory allocation across the board’s 4 GiB of RAM, dynamic mapping and protection, 1000 consecutive 10 ms scheduler sleeps, GPIO output, and repeated warm reloads. Our result provides a functional basis for further Unikraft development on the Raspberry Pi 5. We remain limited to single-core execution, and a small set of peripherals. GPIO still depends on the firmware-configured RP1 BAR and lacks a common Unikraft device interface. Networking, general PCIe devices, PIE, and runtime relocation require further work.

References

1. M. Madson, “Debugging on Raspberry Pi 4 and 5,” January 15, 2024. [Online]. Available: macoy.me. Accessed: August 14, 2026.
2. R. Stange, “Circle: A C++ bare-metal environment for Raspberry Pi with USB (32 and 64 bit),” GitHub repository. [Online]. Available: github.com/rsta2/circle. Accessed: August 14, 2026.
3. U. Agarwal, “HobOS: A hobby OS,” GitHub repository. [Online]. Available: github.com/Utsav-Agarwal/HobOS. Accessed: August 14, 2026.
4. main.lv, “Raspberry 5 bare-metal UART example.” [Online]. Available: main.lv. Accessed: August 14, 2026.
5. Raspberry Pi Ltd., “Raspberry Pi Documentation.” [Online]. Available: raspberrypi.com/documentation. Accessed: August 14, 2026.
6. Unikraft, “Raspberry Pi 3 platform,” GitHub repository. [Online]. Available: github.com/unikraft/plat-raspi. Accessed: August 14, 2026.
7. S. Kuenzer et al., “Unikraft: Fast, Specialized Unikernels the Easy Way,” in *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys '21)*, 2021, pp. 376–394. doi: 10.1145/3447786.3456248.
8. Solo5 Project, “Solo5: A sandboxed execution environment for unikernels,” GitHub repository. [Online]. Available: github.com/Solo5/solo5. Accessed: August 14, 2026.
9. Raspberry Pi Ltd., “I/O controllers: RP1,” *Raspberry Pi Documentation*. [Online]. Available: raspberrypi.com/documentation/computers/io-controllers.html. Accessed: August 14, 2026.
10. Unikraft, “Architecture.” [Online]. Available: unikraft.org/docs/internals/architecture. Accessed: August 14, 2026.
11. Arm Ltd., “Learn the Architecture: AArch64 Exception Model,” document 102412. [Online]. Available: developer.arm.com/documentation/102412/latest. Accessed: August 14, 2026.
12. Raspberry Pi Ltd., “config.txt.” [Online]. Available: raspberrypi.com/documentation/computers/config_txt. Accessed: August 14, 2026.
13. Linux kernel community, “Booting AArch64 Linux.” [Online]. Available: kernel.org/doc/html/latest/arch/arm64/booting.html. Accessed: August 14, 2026.
14. Raspberry Pi Ltd., “Configuration: Device Trees, overlays, and parameters.” [Online]. Available: raspberrypi.com/documentation/computers/configuration.html. Accessed: August 14, 2026.
15. Arm Ltd., “PrimeCell UART (PL011) Technical Reference Manual,” ARM DDI 0183. [Online]. Available: developer.arm.com/documentation/ddi0183/latest. Accessed: August 14, 2026.
16. Arm Ltd., “CoreLink GIC-400 Generic Interrupt Controller Technical Reference Manual,” ARM DDI 0471. [Online]. Available: developer.arm.com/documentation/ddi0471/latest. Accessed: August 14, 2026.
17. Arm Ltd., “AArch64 Programmer’s Guide: Generic Timer,” ARM062-1010708621-30, 2019. [Online]. Available: developer.arm.com. Accessed: August 14, 2026.

18. OpenOCD Project, “GDB and OpenOCD,” OpenOCD User’s Guide. [Online]. Available: openocd.org/doc/html/GDB-and-OpenOCD.html. Accessed: August 14, 2026.
19. Arm Ltd., “Arm Architecture Reference Manual for A-profile architecture,” Arm DDI 0487. [Online]. Available: developer.arm.com. Accessed: August 14, 2026.
20. MirageOS, “MirageOS: A programming framework for building type-safe, modular systems.” [Online]. Available: mirage.io. Accessed: August 14, 2026.
21. IncludeOS, “IncludeOS: Run your application with zero overhead.” [Online]. Available: includeos.org. Accessed: August 14, 2026.
22. A. Kivity, D. Laor, G. Costa, P. Enberg, N. Har’El, D. Marti, and V. Zolotarov, “OSv—Optimizing the Operating System for Virtual Machines,” in *2014 USENIX Annual Technical Conference (USENIX ATC ’14)*, 2014, pp. 61–72. [Online]. Available: usenix.org.
23. Unikraft, “Catalog,” GitHub repository. [Online]. Available: github.com/unikraft/catalog. Accessed: August 15, 2026.
24. Raspberry Pi Ltd., “Raspberry Pi RP1 Peripherals,” document RP-008370-DS-1, November 7, 2023. [Online]. Available: pip-assets.raspberrypi.com. Accessed: August 18, 2026.
25. Raspberry Pi Ltd., “RP1 GPIO chip implementation for pinctrl,” `gpiochip_rp1.c`, Raspberry Pi utilities, GitHub repository. [Online]. Available: github.com/raspberrypi/utls. Accessed: August 18, 2026.
26. Raspberry Pi Ltd. and Linux kernel contributors, “RP1 pin control driver,” `pinctrl-rp1.c`, Raspberry Pi Linux kernel, branch `rpi-6.12.y`, GitHub repository. [Online]. Available: github.com/raspberrypi/linux. Accessed: August 18, 2026.
27. Raspberry Pi Ltd., “Raspberry Pi Debug Probe.” [Online]. Available: raspberrypi.com/documentation/microprobe.html. Accessed: August 26, 2026.
28. L. Urbantat, “plat-raspi5,” GitLab repository. [Online]. Available: gitlab.inf.ethz.ch/PRV-SHINDE/theses/sem_projects/sem-2026/lorin_urbantat/plat-raspi5. Accessed: August 26, 2026.
29. Bruol, “Unikraft,” GitHub repository. [Online]. Available: github.com/Bruol/unikraft. Accessed: August 26, 2026.

A Appendix

A.1 Reproduction Guide

The evaluation used a 4 GiB Raspberry Pi 5, a CMSIS-DAP probe connected to SWD and UART. Connect the debug port on the Raspberry Pi 5 to the debug port on the probe. Connect the UART port on the debug probe to GPIO pins 14 (TXD) and 15 (RXD).

The source repository is available through the ETH Zurich GitLab project [28] and the public GitHub fork [29]. Clone either repository before running the commands in this guide.

The config.txt then sets:

```
enable_jtag_gpio=1
enable_rpi_uart=1
pciex4_reset=0
```

This allows UART and SWD at the same time.

For the first boot, create the boot medium with the whole target device. This operation erases that device.

```
make -f Makefile.rpi5 -j4 flash-usb \
    UK_APP=/absolute/path/to/unikraft/application \
    DEVICE=/dev/{DEVICE}
```

Replace {DEVICE} with the correct removable-disk path; this command overwrites the selected device.

The repository contains six applications under **testapps**:

Directory	Purpose	Expected result
testapps/hello-world	Boot and console smoke test	Prints Hello, World! and returns zero.
testapps/hello-world-cpp	C++ boot and console smoke test	Prints Hello, World! and returns zero.
testapps/eval-mem	Allocation and memory-integrity test	Verifies at least 4075 MiB without corruption.
testapps/dynamic-paging	Runtime page-table test	Maps a page at 2 TiB, checks protection and translation, then unmaps it.
testapps/eval-timer	Interrupt and scheduler-timer test	Completes 1000 sleeps of 10 ms without an early wake-up.
testapps/rpi5-gpio	RP1 GPIO output test	Blinks an LED connected to GPIO23 ten times.

Select an application by passing its directory as **UK_APP**. For example, build and verify the GPIO test with:

```
make -f Makefile.rpi5 verify -j4 \
    UK_APP="$(pwd)/testapps/rpi5-gpio"
```

Use the same command with one of the other directories to reproduce the C hello-world, C++ hello-world, memory, or timer evaluation. You can also supply an application from Unikraft's catalog [23] or another Unikraft application through `UK_APP`.

For the GPIO test, connect an LED and a suitable series resistor between GPIO23 and ground. Load the image and continue execution through GDB. A successful run prints alternating `HIGH` and `LOW` messages while the LED blinks ten times. The test holds each state for two seconds.

Connect UART at 115200 baud and start OpenOCD and GDB in separate terminals:

```

TERMINAL 1
tio /dev/cu.usbmodem102 -b 115200
TERMINAL 2
sudo openocd --file debug_cfg/cmsis-dap.cfg --file debug_cfg/openocd_raspi5.cfg
TERMINAL 3
gdb -q -x debug_cfg/ConnectJTAG.gdb
```

The serial-device and USB-device names depend on the host. In GDB, run `continue` and observe the UART output.

Initially the kernel waits for GDB to attach and continue. This behavior happens only if `CONFIG_RPI5_EARLY_GDB_SPINLOOP=y` is set in `rpi5_config`. Otherwise the kernel boots without waiting. A successful run prints `Hello, World!`.

To reproduce the warm-reload result, leave the board powered, quit GDB, and run:

```
make -j8 && gdb -q -x debug_cfg/ConnectJTAG.gdb
```

This compiles the kernel, loads it into ram using SWD, runs the reset trampoline on the pi, and sets the `$pc` to the address that contains the new kernel.

A.2 Debugging Workflow

We used a tmux session during development with panes for OpenOCD, GDB, and UART. The Benefit of this is that you can point a coding agent at the tmux session. It can then automatically inspect the live hardware and run experiments on the hardware rapidly. This enables ultra fast debugging and validating hypotheses.

After the first cold boot, the resident trampoline let GDB disable the MMU and caches, load a new kernel and DTB, and restart without rewriting the boot medium or power-cycling. Most iterations of the kernel required only a rebuild and `rpi5_reload`.

Use of Artificial Intelligence

Artificial intelligence tools were used to aid with research, coding, and writing during this project. Their output was critically reviewed, adapted, and verified by the author, who remains responsible for the content and results of this work.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Eigenständigkeitserklärung

Die unterzeichnete Eigenständigkeitserklärung ist Bestandteil jeder während des Studiums verfassten schriftlichen Arbeit. Eine der folgenden zwei Optionen ist **in Absprache mit der verantwortlichen Betreuungsperson** verbindlich auszuwählen:

- ☐ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe, namentlich, dass mir niemand beim Verfassen der Arbeit geholfen hat. Davon ausgenommen sind sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson. Es wurden keine Technologien der generativen künstlichen Intelligenz¹ verwendet.
- ☒ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe. Dabei habe ich nur die erlaubten Hilfsmittel verwendet, darunter sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson sowie Technologien der generativen künstlichen Intelligenz. Deren Einsatz und Kennzeichnung ist mit der Betreuungsperson abgesprochen.

Titel der Arbeit:

Porting Unikraft to Raspberry Pi 5

Verfasst von:

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich.

Name(n):

Urbantat

Vorname(n):

Richard Lorin

Ich bestätige mit meiner Unterschrift:

- Ich habe mich an die Regeln des «[Zitierleitfadens](#)» gehalten.
- Ich habe alle Methoden, Daten und Arbeitsabläufe wahrheitsgetreu und vollständig dokumentiert.
- Ich habe alle Personen erwähnt, welche die Arbeit wesentlich unterstützt haben.

Ich nehme zur Kenntnis, dass die Arbeit mit elektronischen Hilfsmitteln auf Eigenständigkeit überprüft werden kann.

Ort, Datum

Unterschrift(en)

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich. Durch die Unterschriften bürgen sie grundsätzlich gemeinsam für den gesamten Inhalt dieser schriftlichen Arbeit.

¹ Für weitere Informationen konsultieren Sie bitte die Webseiten der ETH Zürich, bspw. <https://ethz.ch/de/die-eth-zuerich/lehre/ai-in-education.html> und <https://library.ethz.ch/forschen-und-publizieren/Wissenschaftliches-Schreiben-an-der-ETH-Zuerich.html> (Änderungen vorbehalten).